

RP 서버 API 쿼리 레퍼런스

앱/웹 클라이언트 → RP 서버 API 요약. 빠른 조회용 — 배경 설명·구현 가이드는 [rp-client-api.md](#) 참조.

- **Base URL:** `https://rp.example.com` (데모 `http://localhost:8090`)
- **응답:** 전 엔드포인트 `ApiResponse` envelope. 성공 → `data`, 실패 → `code` / `error`.
- **경로 주의:** ceremony 5종(`/passkey/**`)은 SDK 고정. 회원·자격증명(`/users` · `/me` · `/credentials`)은 RP 자체 구현 → 경로 다를 수 있음(데모 기준).

엔드포인트 일람

#	Method	Path	인증	설명
1	POST	<code>/users</code>	—	회원 생성 → <code>externalUserId</code> 발급
2	POST	<code>/passkey/register/begin</code>	—	등록 옵션(challenge) 발급
3	POST	<code>/passkey/register/finish</code>	—	등록 결과 검증·저장
4	POST	<code>/passkey/authenticate/begin</code>	—	인증 옵션(challenge) 발급
5	POST	<code>/passkey/authenticate/finish</code>	—	인증 검증 → 토큰 발급
6	POST	<code>/passkey/refresh</code>	—	토큰 갱신
7	GET	<code>/me</code>	Bearer	내 정보 조회
8	GET	<code>/credentials</code>	Bearer	내 패스키 목록
9	PATCH	<code>/credentials/{id}</code>	Bearer	패스키 이름 변경
10	DELETE	<code>/credentials/{id}</code>	Bearer	패스키 삭제

인증 열: — 불필요 / Bearer = `Authorization: Bearer <accessToken>` 필수.

공통 응답 Envelope

```
// 성공
{ "success": true, "code": "OK", "message": "...", "data": {...},
  "traceId": "...", "timestamp": "..."}
// 실패
{ "success": false, "code": "P003", "message": "...",
  "error": { "errorCode": "P003", "fieldErrors": null },
  "traceId": "...", "timestamp": "..."}
}
```

아래 각 엔드포인트의 **Response**는 envelope의 `data` 필드 내용만 표기한다.

흐름 한눈에

[등록] 1 /users → 2 register/begin → navigator.credentials.create() → 3 register/finish
 [로그인] 4 authenticate/begin → navigator.credentials.get() → 5 authenticate/finish → accessToken
 [유지] accessToken 만료 → 6 refresh 보호 API(7~10) → Authorization: Bearer

1. POST /users — 회원 생성

	필드	타입	비고
Request	displayName	string	표시 이름
Response	externalUserId	string	RP 사용자 식별자 (이후 ceremony에 사용)
	displayName	string	

2. POST /passkey/register/begin — 등록 옵션 발급

	필드	타입	비고
Request	externalUserId	string	1번 응답값
	displayName	string	
Response	ceremonyId	string(UUID)	3번 요청에 재전달
	challenge	string(B64URL)	create() 에 디코딩해 전달
	rp	object	{ id, name }
	user	object	{ id(B64URL), name, displayName }
	pubKeyCredParams	array	[{ type, alg }]
	timeout	number	ms
	attestation	string	none 등
	authenticatorSelection	object	{ userVerification, residentKey, requireResidentKey }
	excludeCredentials	array	[{ type, id(B64URL), transports }] (없으면 생략)

3. POST /passkey/register/finish — 등록 완료

	필드	타입	비고
Request	ceremonyId	string(UUID)	2번 응답값
	credentialId	string	cred.id
	clientDataJsonB64u	string(B64URL)	cred.response.clientDataJSON
	attestationObjectB64u	string(B64URL)	cred.response.attestationObject
	transports	string	getTransports() 콤마 join
	nickname	string	기기 라벨 (선택)
Response	credentialDbId	string(UUID)	
	credentialId	string	
	aaguid	string	authenticator 모델 ID

4. POST /passkey/authenticate/begin — 인증 옵션 발급

	필드	타입	비고
Request	externalUserId	string	(생략 시 discoverable 흐름)
Response	ceremonyId	string(UUID)	5번 요청에 재전달
	challenge	string(B64URL)	get() 에 디코딩해 전달
	timeout	number	ms
	rpId	string	
	allowCredent- tials	array	[{ type, id(B64URL), transports }]
	userVerifica- tion	string	preferred 등

5. POST /passkey/authenticate/finish — 인증 완료

	필드	타입	비고
Request	ceremonyId	string(UUID)	4번 응답값
	credentialId	string	assertion.id
	clientDataJsonB64u	string(B64URL)	assertion.response.clientData-JSON
	authenticatorData-B64u	string(B64URL)	assertion.response.authenticatorData
	signatureB64u	string(B64URL)	assertion.response.signature
	userHandleB64u	string(B64URL)	assertion.response.userHandle (null 가능)
Response	credentialDbId	string(UUID)	
	tenantUserId	string(UUID)	
	credentialId	string	
	signatureCounter	number	
	accessToken	string(JWT)	보호 API Bearer 에 사용
	refreshToken	string(JWT)	6번 갱신에 사용
	accessExpiresIn	number	엑세스 토큰 유효 초

6. POST /passkey/refresh — 토큰 갱신

	필드	타입	비고
Request	refreshToken	string(JWT)	직전 발급값
Response	accessToken	string(JWT)	새 값
	refreshToken	string(JWT)	새 값 — 반드시 교체 저장
	accessExpiresIn	number	

`refreshToken`은 1회용. 이전 값 재사용 시 토큰 패밀리 전체 무효화 → 재로그인.

7. GET /me — 내 정보

Authorization: Bearer <accessToken> 필수.

	필드	타입
Response	<code>externalUserId</code>	string
	<code>displayName</code>	string

8. GET /credentials — 내 패스키 목록

Authorization: Bearer <accessToken> 필수. **Response:** 아래 객체의 배열.

필드	타입	비고
<code>id</code>	string(UUID)	9·10번 <code>{id}</code> 에 사용
<code>credentialId</code>	string	
<code>nickname</code>	string	기기 라벨
<code>status</code>	string	ACTIVE 등
<code>aaguid</code>	string	authenticator 모델 ID
<code>transports</code>	string	
<code>signatureCounter</code>	number	
<code>lastUsedAt</code>	string(ISO8601)	
<code>createdAt</code>	string(ISO8601)	
<code>revokedAt</code>	string(ISO8601)OrNull	
<code>revokedReason</code>	stringOrNull	

9. PATCH /credentials/{id} — 패스키 이름 변경

Authorization: Bearer <accessToken> 필수. {id} = 8번 응답의 id.

	필드	타입
Request	nickname	string
Response	—	변경된 credential 객체 (8번 항목과 동일)

10. DELETE /credentials/{id} — 패스키 삭제

Authorization: Bearer <accessToken> 필수. {id} = 8번 응답의 id.

Response: 200 + envelope (success: true, data 없음).

에러 코드

success: false 이면 code 로 분기. prefix — C/P 플랫폼, D RP 서버.

코드	HTTP	의미	대응
C001	400	입력값 검증 실패	error.fieldErrors 확인
P001	404	테넌트 WebAuthn 설정 없음	운영팀 문의
P003	404	challenge 만료/없음	begin부터 재시도
P004	401	서명 검증 실패	다른 패스키로 재시도
P005	401	signature counter 이상	해당 패스키 사용 중단 권고
P006	403	자격증명 소유권 불일치	본인 자격증명만 관리 가능
D001	401	인증 필요 (토큰 누락/무효)	refresh → 실패 시 재로그인
D002	404	사용자 없음	회원가입 흐름 재진입
—	429	rate limit 초과	잠시 후 재시도

전체 코드: [error-codes.md](#).

클라이언트 주의사항 (요약)

- `navigator.credentials.create/get` 은 사용자 제스처 안에서 호출.
- `*B64u` 필드 `challenge` · `*.id` · `userHandle` 은 **Base64URL** 인코딩.
- `ceremonyId` 는 begin → finish 사이 보관.
- `refreshToken` 은 갱신 때마다 새 값으로 교체 저장.
- WebAuthn 미지원 감지: `window.PublicKeyCredential` 존재 확인.
- 사용자 취소(`NotAllowedError`)와 실제 오류 구분.

브라우저 웹: `@crosscert/passkey-sdk` 가 ceremony·인코딩 래핑 ([integration-guide.md](#)). 네이티브 앱: OS 패스키 API(iOS `ASAuthorization` / Android `Credential Manager`)로 `create/get` 대체, HTTP 호출은 동일.